Project: CIFAR-10 Image Classification using Convolutional Neural Networks and Logistic Regression

Author: Vraj Diyora

Abstract: In this project, we explore the application of Convolutional Neural Networks (CNN) and Logistic Regression for image classification on the CIFAR-10 dataset. The performance of these two supervised learning models and compare their accuracy, computational efficiency, and suitability for image classification tasks.

Introduction: The CIFAR-10 dataset comprises 60,000 32x32 color images across 10 classes, with 6,000 images per class. Our objective is to develop accurate classification models capable of correctly identifying images into their respective classes. We explore CNNs, well-suited for image classification tasks due to their ability to capture spatial hierarchies of features, and logistic regression, a simple yet effective linear classifier.

Convolutional Neural Network (CNN):

1. **Importing Libraries:** We utilize Keras, TensorFlow, and the ImageDataGenerator for data preprocessing.
2. **Building the CNN:** We construct a sequential CNN model consisting of convolutional layers, max-pooling layers, a flattening layer, fully connected layers, and an output layer.
3. **Compiling and Training the CNN:** We compile the model using the Adam optimizer, categorical cross-entropy loss function, and accuracy metric. CNN is trained on the CIFAR-10 training dataset with specified epochs and batch sizes.
4. **Evaluating the CNN:** We evaluate the trained CNN model on the test dataset and report the test loss and accuracy.

Logistic Regression:

1. **Importing Libraries:** We import necessary libraries including sci-kit-learn for logistic regression.
2. **Data Preprocessing:** We flatten the CIFAR-10 images and standardize the features.
3. **Defining the Logistic Regression Model:** We define a logistic regression model using scikit-learn's LogisticRegression class.
4. **Training the Logistic Regression Model:** We train the logistic regression model on the training dataset.
5. **Evaluating the Logistic Regression Model:** We evaluate the trained model on the test dataset and report the accuracy.

Model Architectures:

1. **Logistic Regression Model:**
    - Input Layer: Flatten layer to reshape input images into a vector.
    - Output Layer: Dense layer with 100 units (equal to the number of classes) and softmax activation function.
    - Total Trainable Parameters: 307,300
2. **Convolutional Neural Network (CNN) Model:**
    - Convolutional Layers: Three convolutional layers with 32, 64, and 128 filters respectively, each followed by max-pooling layers.
    - Dense Layers: Two fully connected dense layers with 128 and 100 units respectively, followed by softmax activation.
    - Total Trainable Parameters: 1,175,852

Results:

1. **Logistic Regression Model:**
    - Training Accuracy: 10.89%
    - Validation Accuracy: 10.22%
    - Test Accuracy: 10.43%
    - Test Loss: 4.1843
2. **Convolutional Neural Network (CNN) Model:**
    - Training Accuracy: 55.01%
    - Validation Accuracy: 38.32%
    - Test Accuracy: 39.93%
    - Test Loss: 2.4789

Comparison:

- **Accuracy:** CNNs generally outperform logistic regression due to their ability to capture complex hierarchical features in images.
- **Computational Efficiency:** Logistic regression is computationally less intensive compared to CNNs, making it suitable for simpler datasets or scenarios where computational resources are limited.

- **Interpretability:** Logistic regression provides interpretable results and insights into the importance of individual features, which can be beneficial for understanding the underlying factors driving classification decisions.

Conclusion: In conclusion, both CNNs and logistic regression offer viable approaches for image classification on the CIFAR-10 dataset. While CNNs excel in capturing intricate features and achieving higher accuracy, logistic regression provides a simpler and more interpretable alternative. The choice between these models depends on factors such as the complexity of the dataset, computational resources, and the interpretability of results. Further experimentation and tuning may be necessary to optimize the performance of each model for specific use cases.

```
#final project (cifar100)
#Author:"Vraj Diyora"
```

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
[16] # dependencies
     import keras
     from keras.datasets import cifar100
     import numpy as np
     from tensorflow.keras.models import Sequential
     from tensorflow.keras.layers import Flatten, Dense
     from sklearn.svm import SVC
     from sklearn.metrics import accuracy_score
```

```
[3]

     # define num_class
     num_classes = 100

     # load dataset keras will download cifar-100 datset
     (x_train, y_train), (x_test, y_test) = cifar100.load_data()
```

```python
# define num_class
num_classes = 100

# load dataset keras will download cifar-100 datset
(x_train, y_train), (x_test, y_test) = cifar100.load_data()

print('x_train shape:', x_train.shape)
print(x_train.shape[0], 'train samples')
print(x_test.shape[0], 'test samples')
```

```
x_train shape: (50000, 32, 32, 3)
50000 train samples
10000 test samples
```

[17]
```python
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

```python
# Flatten the images
x_train_flat = x_train.reshape((len(x_train), -1))
x_test_flat = x_test.reshape((len(x_test), -1))
```

```
[ ] # Define Logistic Regression model
    lr_model = Sequential([
        Flatten(input_shape=(32, 32, 3)),
        Dense(100, activation='softmax')
    ])

    # Compile the model
    lr_model.compile(optimizer='adam',
                     loss='sparse_categorical_crossentropy',
                     metrics=['accuracy'])

    # Train the model
    lr_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.1)

    # Evaluate the model
    test_loss, test_accuracy = lr_model.evaluate(x_test, y_test)
    print('Test Loss:', test_loss)
    print('Test Accuracy:', test_accuracy)
```

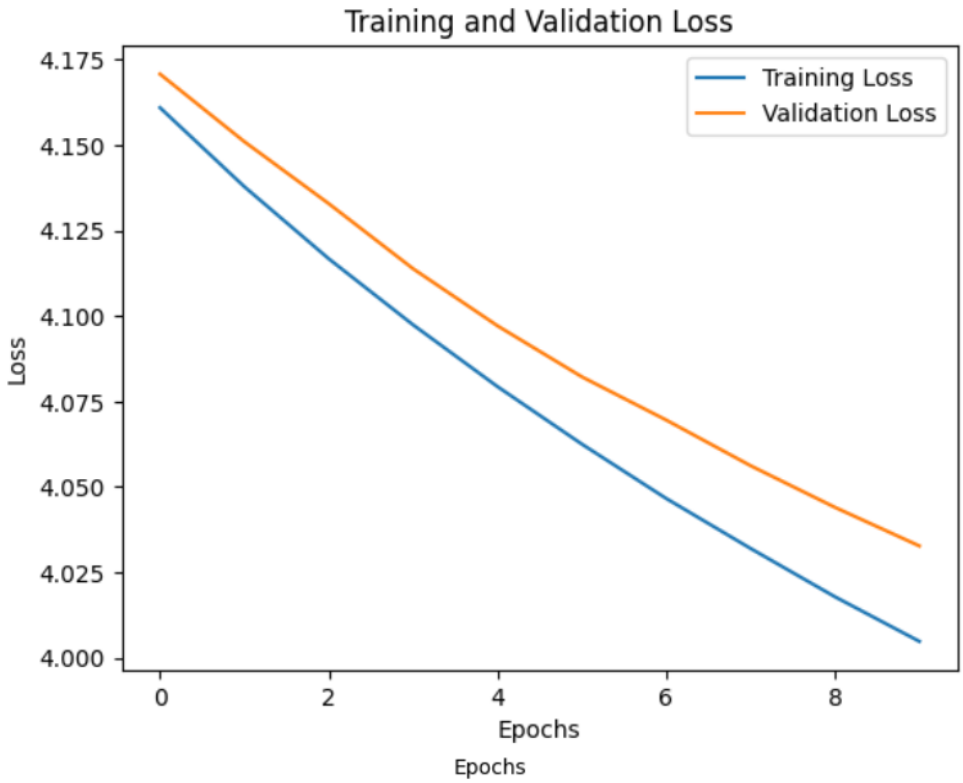
```
Epoch 1/10
704/704 [==============================] - 7s 9ms/step - loss: 4.5698 - accuracy: 0.0172 - val_loss: 4.5375 - val_accuracy: 0.0268
Epoch 2/10
704/704 [==============================] - 7s 9ms/step - loss: 4.5058 - accuracy: 0.0434 - val_loss: 4.4797 - val_accuracy: 0.0610
Epoch 3/10
704/704 [==============================] - 6s 9ms/step - loss: 4.4492 - accuracy: 0.0673 - val_loss: 4.4283 - val_accuracy: 0.0766
Epoch 4/10
704/704 [==============================] - 5s 7ms/step - loss: 4.3986 - accuracy: 0.0770 - val_loss: 4.3828 - val_accuracy: 0.0820
Epoch 5/10
704/704 [==============================] - 10s 14ms/step - loss: 4.3533 - accuracy: 0.0881 - val_loss: 4.3426 - val_accuracy: 0.0856
Epoch 6/10
704/704 [==============================] - 9s 13ms/step - loss: 4.3125 - accuracy: 0.0909 - val_loss: 4.3068 - val_accuracy: 0.0944
Epoch 7/10
704/704 [==============================] - 6s 9ms/step - loss: 4.2763 - accuracy: 0.0980 - val_loss: 4.2733 - val_accuracy: 0.0940
Epoch 8/10
704/704 [==============================] - 5s 7ms/step - loss: 4.2428 - accuracy: 0.1010 - val_loss: 4.2446 - val_accuracy: 0.0894
Epoch 9/10
704/704 [==============================] - 5s 7ms/step - loss: 4.2131 - accuracy: 0.1041 - val_loss: 4.2184 - val_accuracy: 0.0976
Epoch 10/10
704/704 [==============================] - 9s 13ms/step - loss: 4.1857 - accuracy: 0.1089 - val_loss: 4.1942 - val_accuracy: 0.1022
313/313 [==============================] - 2s 7ms/step - loss: 4.1843 - accuracy: 0.1043
Test Loss: 4.184309959411621
Test Accuracy: 0.10429999977350235
```

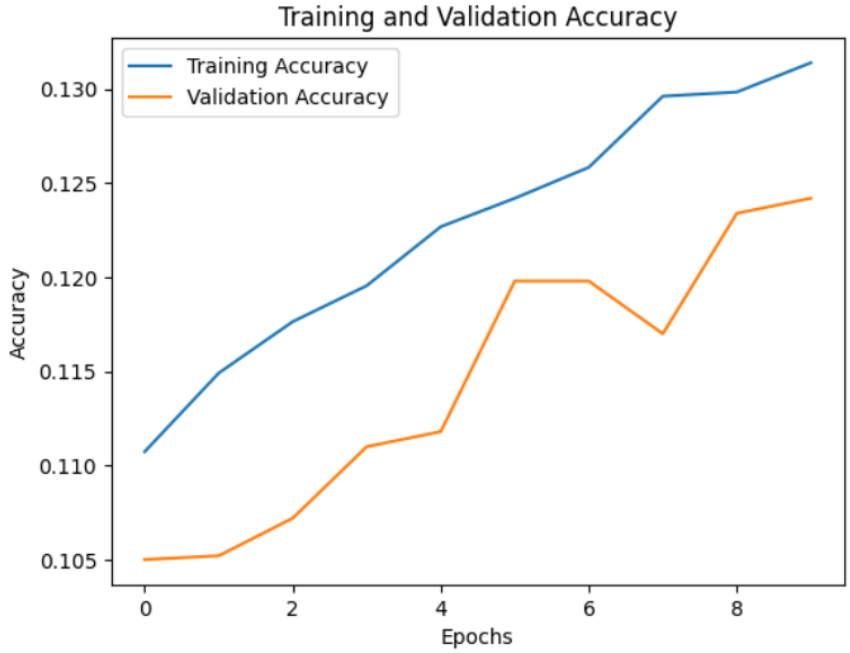
```
import matplotlib.pyplot as plt

# Train the model
history = lr_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.1)

# Plot training and validation loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```


Training and Validation Loss
Training Loss
Validation Loss
Loss
4.175
4.150
4.125
4.100
4.075
4.050
4.025
4.000
0
2
4
6
8
Epochs
Epochs


Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
0.130
0.125
0.120
0.115
0.110
0.105
0
2
4
6
8
Epochs

17m

```python
import keras
from keras.datasets import cifar100
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Load CIFAR-100 dataset
(x_train, y_train), (x_test, y_test) = cifar100.load_data()

# Preprocess the data
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Convert labels to one-hot encoding
num_classes = 100
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)

# Define CNN model
cnn_model = Sequential([
    Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(num_classes, activation='softmax')
])

# Compile the model
cnn_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

# Train the model
cnn_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate the model
test_loss, test_accuracy = cnn_model.evaluate(x_test, y_test)
print('Test Loss:', test_loss)
print('Test Accuracy:', test_accuracy)
```

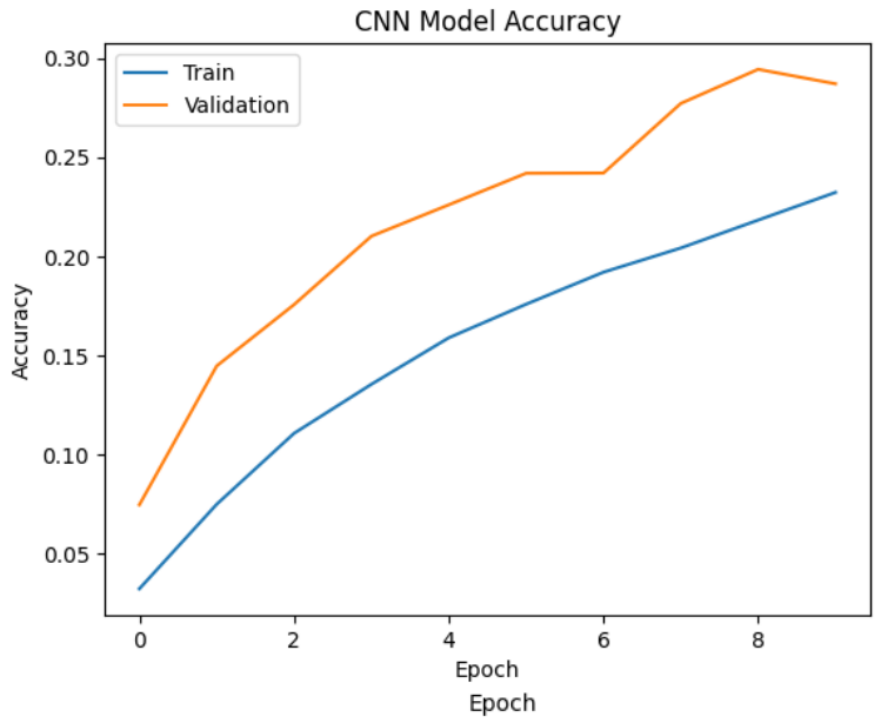
```
Epoch 1/10
704/704 [==============================] - 106s 149ms/step - loss: 3.9057 - accuracy: 0.1068 - val_loss: 3.4600 - val_accuracy: 0.1800
Epoch 2/10
704/704 [==============================] - 99s 141ms/step - loss: 3.1582 - accuracy: 0.2292 - val_loss: 3.0217 - val_accuracy: 0.2596
Epoch 3/10
704/704 [==============================] - 104s 147ms/step - loss: 2.7741 - accuracy: 0.3040 - val_loss: 2.7895 - val_accuracy: 0.3000
Epoch 4/10
704/704 [==============================] - 112s 159ms/step - loss: 2.5081 - accuracy: 0.3560 - val_loss: 2.6471 - val_accuracy: 0.3320
Epoch 5/10
704/704 [==============================] - 105s 149ms/step - loss: 2.2954 - accuracy: 0.4020 - val_loss: 2.5756 - val_accuracy: 0.3516
Epoch 6/10
704/704 [==============================] - 105s 149ms/step - loss: 2.1348 - accuracy: 0.4366 - val_loss: 2.4910 - val_accuracy: 0.3732
Epoch 7/10
704/704 [==============================] - 104s 148ms/step - loss: 1.9912 - accuracy: 0.4677 - val_loss: 2.4126 - val_accuracy: 0.3872
Epoch 8/10
704/704 [==============================] - 104s 148ms/step - loss: 1.8610 - accuracy: 0.4980 - val_loss: 2.4677 - val_accuracy: 0.3808
Epoch 9/10
704/704 [==============================] - 101s 144ms/step - loss: 1.7403 - accuracy: 0.5251 - val_loss: 2.4773 - val_accuracy: 0.3914
Epoch 10/10
704/704 [==============================] - 103s 146ms/step - loss: 1.6315 - accuracy: 0.5501 - val_loss: 2.5094 - val_accuracy: 0.3832
313/313 [==============================] - 6s 20ms/step - loss: 2.4789 - accuracy: 0.3993
Test Loss: 2.4789462089538574
Test Accuracy: 0.3993000090122223
```

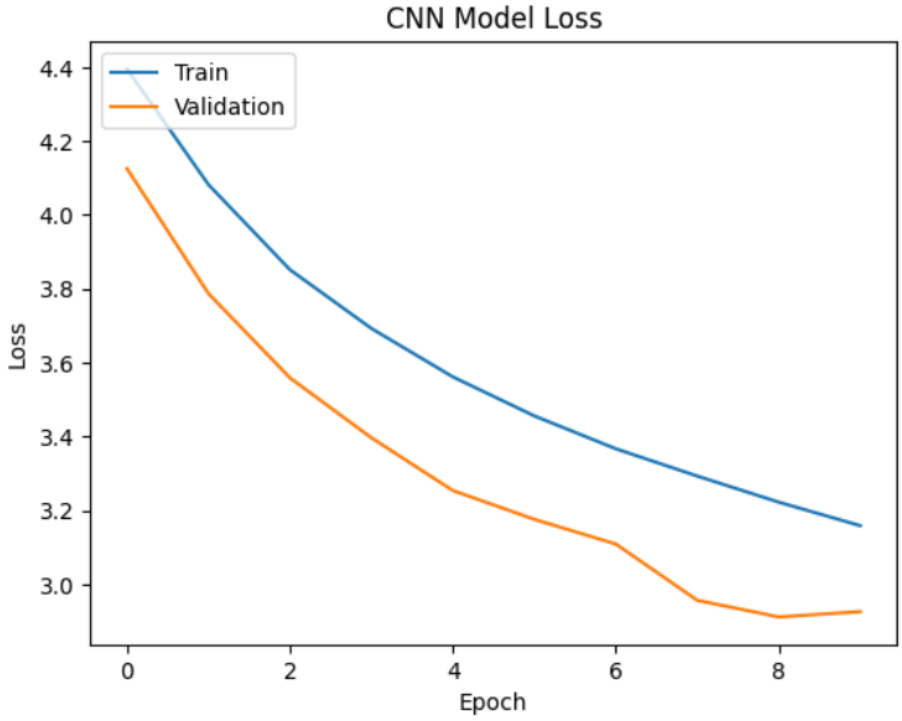
0s

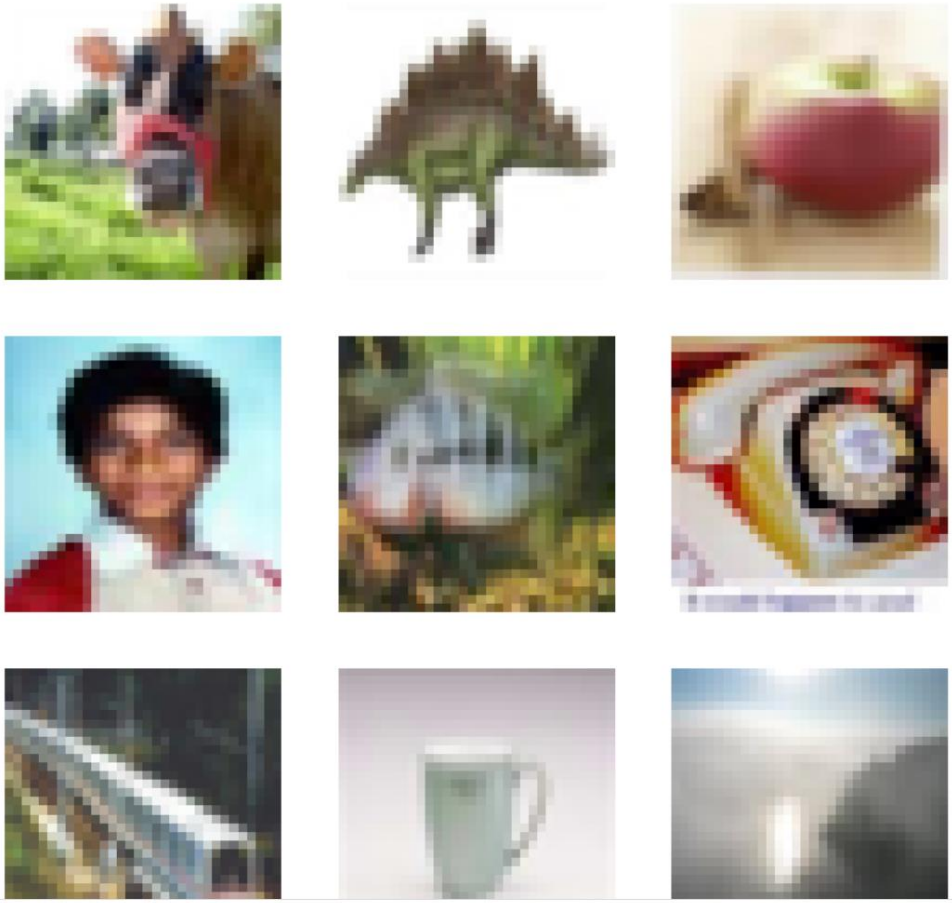
```python
import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('CNN Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('CNN Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```


CNN Model Accuracy
Train
Validation
Accuracy
0.30
0.25
0.20
0.15
0.10
0.05
0
2
4
6
8
Epoch
Epoch


CNN Model Loss
Train
Validation
Loss
4.4
4.2
4.0
3.8
3.6
3.4
3.2
3.0
0
2
4
6
8
Epoch

```python
# Display sample images
plt.figure(figsize=(10, 10))
for i in range(9):
    plt.subplot(3, 3, i+1)
    plt.imshow(x_train[i])
    plt.axis('off')
plt.suptitle('Sample Images')
plt.show()
```

Sample Images